2022

Memòria Pràctica 2

04-03-2022

CRISTIAN PRADO MÉNDEZ I ALBA CANO IGLESIAS

ÍNDEX:

1. **Introducció.**
2. **Qüestions a respondre.**
   - **2.1. Explicar les classes implementades .**
   - **2.2. Explicar com has declarat l'atribut de la classe ClubUB i perquè.**
   - **2.3. Dibuixar el diagrama de relacions entre les classes que heu utilitzat a la vostra pràctica. No cal incloure la llista d'atributs i mètodes.**
   - **2.4. Explicar quins són els atributs de la classe Soci i perquè.**
   - **2.5. Per què creieu que la classe Soci ha de ser abstracta?**
   - **2.6. Creieu que podríem haver treballat només amb dues classes Soci i SociFederat? Per què creus que és necessari crear les tres classes Soci, SociEstandard, SociFederat i SociJunior?**
   - **2.7. Indiqueu si hi ha i on es troben els exemples de mètodes polimòrfics al vostre codi.**
   - **2.8. Analitzeu perquè l'objecte de tipus Asseguranca es crea fora del constructor de la classe SociEstandard. Es podria instanciar a dins? Com canviaria el sentit de la implementació?**
   - **2.9. Expliqueu com heu utilitzat la classe ExcepcioClub al vostre codi.**
   - **2.10. 2.10 Si teniu un soci federat que fa 5 excursions en un mes quina serà la quantitat de la seva factura aquell mes?**
   - **2.11 Si teniu un soci estàndard amb una assegurança de tipus Bàsica d'un preu de 5€ i que fa 5 excursions en un mes quina serà la quantitat de la seva factura aquell mes?**
   - **2.11. 2.12 Detallar les proves realitzades per comprovar el correcte funcionament de la pràctica, resultats obtinguts i accions derivades.**
   - **2.12. 2.13 Observacions generals.**
3. **Resultats.**

1. Introducció.

En aquesta part de la memòria tractarem d'explicar quin són els objectius, així com els continguts que s'han vist en la realització de la mateixa.

En aquesta pràctica la nostre tasca era realitzar un programa que permetés la gestió correcta d'un club excursionista en vers els seus socis. Fent ús del paradigma de la programació orientada a objectes hem aconseguit desenvolupar un programa que compleix amb totes les funcionalitats establertes com a objectius:

- Afegir socis en funció del tipus.
- Mostrar llistes de socis (en funció del tipus).
- Eliminar i verificar si les dades d'un soci són correctes.
- Mostrar la factura segons el soci.
- Modificar tan el nom com el tipus d'assegurança d'un soci (estàndard o junior).
- Guardar i recuperar dades.

En la resolució de la pràctica hem tractat diferents aspectes de la programació orientada a objectes, com ara:

- Conceptes fonamentals: classes i objectes al llarg de tota la pràctica.
- Ús de constructors.
- Ús d'interfícies.
- Herència i jerarquia de classes, a la classe Soci i les seves derivades.
- Polimorfisme, en els mètodes de les classes filles de Soci.
- Casting per accedir-hi als mètodes definits a les subclasses de la classe Soci.
- ArrayList.
- Gestió i ús dels fitxers.
- Ús d'excepcions.

2. Qüestions a respondre.

2.1 Explicar les classes implementades.

<u>IniciadorClubUB:</u> Aquesta classe conté el main() encarregat de llençar el bucle principal de tota l'aplicació. Per fer-ho el main() conté un objecte de tipus vistaClubUB que crida al seu mètode gestioClub(), qui conté el bucle.

<u>VistaClubUB:</u> Aquesta classe és la responsable de la interacció entre l'usuari i el propi programa. Per fer-ho té com a atribut una instància de la classe ClubUB que conté tota la informació dels socis.

Aquesta classe depèn de la classe Menu, per tal de poder fer una correcta gestió del bucle principal de l'aplicació. Mitjançant un ENUM es declaren les diferents opcions que es mostraran per pantalla quan es cridi al mètode mostraMenu() (de la classe Menu) dins el mètode de la pròpia classe gestióClub().

Dins gestioClub() trobem el bucle principal. Aquest conté un switch on es criden a tots els mètodes necessaris amb la fi d'aconseguir que cadascuna de les funcionalitats proposades a l'inici de la pràctica s'executin correctament. Cal destacar que és en aquest bucle on es fa el tractament de les excepcions, mitjançant un try-catch que comprova tot el switch.

A més a més, dins el switch principal també trobem un altre switch, que executa un segon menú per tal de poder donar d'alta als socis en funció del tipus.

ClubUB: La classe ClubUB és la classe que conté tota la informació sobre el club (informació sobre tots els socis que pertanyen al club). Està lligada a la classe del codi base VistaClubUB i aquesta demana informació sobre el club.

ClubUB té com a atributs una instància de la classe LlistaSocis, creada pel constructor de la pròpia classe, entre altres que s'expliquen en l'apartat següent d'aquest mateix exercici. A partir d'aquest objecte la classe implementa diferents mètodes que permeten la correcta gestió dels socis.

Per una part, trobem el getter i el setter de la variable nom, que ens serveix per donar-li un nom al club. Seguidament, trobem tres mètodes molt similars que s'encarreguen de crear socis en funció dels seus tipus i cridar el mètode addSoci que els fica dins l'ArrayList de la classe LlistaSocis que conté a tots els socis del club.

D'altra banda, trobem mètodes gestors com llistarSocis(String tipus) el qual crida al mètode de mateix nom de la classe LlistaSocis que retorna un string amb tots els socis i tota la informació relativa a cadascun d'ells. RemoveSoci(int position) que esborra un soci de la llista en funció d'una posició demanada a l'usuari des de la classe VistaClubUB. Un altre mètode és verificarSocis(), encarregat de llençar excepcions en cas que la informació d'algún soci sigui incorrecte. A més a més, tenim mostrarFacturaTotal(String dni, int excursions) que calcula la factura en funció del soci. Per últim, en aquest apartat, trobem els mètodes guardarDades() i recuperarDades() els quals treballen amb fitxers. Aquests s'encarreguen de guardar les dades de la llista dins un fitxer que es crea automàticament en cridar el mètode i de recuperar les dades ja introduïdes prèviament un cop s'executa el mètode respectivament.

Finalment, trobem mètodes amb els que l'usuari pot variar la informació dels socis com ara modificarNomSoci, que tal i com el seu nom indica permet a l'usuari canviar el nom d'un soci concret introduint el seu DNI i modificarTipusAsseguranca(String dni, String assegurança, float preuAsseguranca) que permet modificar tan el tipus com el preu de l'assegurança d'un soci concret introduint, també, el seu DNI.

Tal i com s'indica a l'enunciat de la pràctica, cal destacar, que aquesta classe implementa la interfície Serializable per tal de poder treballar amb els fitxers guardant-hi els objectes necessaris.

Soci: La classe Soci ha de guardar tota la informació dels socis. Aquesta és una classe que implementa dues interfícies. En primer lloc, implementa a InSoci on es troben declarats el mètodes que seran implementats en la pròpia classe i que a continuació explicarem. Així com, a la interfície Serializable, implementació necessària per poder guardar objectes d'aquest tipus en els fitxers generats en la classe ClubUB (tal i com s'ha explicat anteriorment).

Abans d'explicar els mètodes que trobem en aquesta classe, cal destacar que és una classe abstracte que actua com a mare de les següents classes: SociFederat i SociEstandard (la qual alhora és mare de SociJunior). No explicxarem aquí perquè ha de ser abstracta, ja que s'explicarà en l'apartat 2.5. Tampoc explicarem els seus atributs doncs queda explicat en apartats posteriors, més concretament en l'apartat 2.4.

En la classe Soci trobem els getters i els setters dels seus atributs que s'encarreguen retornar i assignar-lis el nom i el dni als diferents socis. Així com la declaració dels mètodes toString(), getTipus(), calculaQuota (floatQuotaBase), calculaPreuExcursio (float preuExcursioBase) i comprova(), tots mètodes abstractes que s'implementen en les seves respectives classes filles.

SociFederat: Aquesta classe actua com a classe filla de Soci, per tant, hereta tots els atributs i mètodes de l'última. En el seu constructor es reben els mateixos paràmetres que en la classe Soci a més de quatre nous atributs que hem d'afegir:

```
private Federacio federacio;
private float descompte_excursions_federats;
private float descompte_quota_federats;
private String _tipus = "Federat";
```

Tots aquests atributs es troben inicialitzats al constructor de la pròpia classe. A més, la classe implementa els mètodes abstracts mencionats en el punt anterior.

El mètode toString() s'encarrega de retornar un String amb tots els detalls del soci Federat. El mètode getTipus() retorna el tipus de soci. També s'implementen els mètodes calculaQuota(float quota Base) que retorna un float amb el preu de la quota aplicant el descompte adient. Així com el mètode ClaculaPreuExcursio(float preuExcursioBase) que també retorna un float amb el preu d'una excursió un cop aplicat el descompte adient. Per últim trobem el mètode comprova() que verifica si el preu de la federació és correcte. En cas de no ser-ho llença una excepció.

SociEstandard: La classe SociEstàndard és molt similar a la classe SociFederat, però en comptes de tenir atributs de la classe Feredario, presenta un atribut de la classe Asseguranca.

```
private String _tipus = "Estàndard";
protected Asseguranca asseguranca;
```

En essència els mètodes que conté aquest aclasse són exactament iguals que en la classe SociFederat, amb les diferencies de que en els mètodes calculaQuota(float quota Base) i CalculaPreuExcursio(float preuExcursioBase) no s'aplica cap mena de descompte i el mètode comprova verifica si es tipus d'assegurança és o no correcte alhora de llençar una excepció.

A més a més, aquesta classe no només actua com a classe filla de Soci, sinó que també actua com a classe mare de SociJunior.

<u>SociJunior:</u> SociJunior actua com a classe filla de SociEstandard tal i com ja s'ha explicat, per tant conté tots els atributs i mètodes que aquesta. En el seu constructor es reben els mateixos paràmetres que en la classe SociEstandard, i dins del seu constructor es crida al super.

A més també declarem un nou atribut únic d'aquest tipus de soci, l'any de naixement. Per tant, a més dels mètodes que hereta, aquesta classe també implementa el getter i el setter del nou atribut que retorna i assigna l'any de naixement al soci. Així com els mètodes toString(), getTipus() i calculaPreuExcursio(float preuExcursioBase) (que retorna un cero, ja que el soci Junior no ha de pagar) que presenten un polimorfisme.

<u>LlistaSocis:</u> La classe LlistaSocis s'encarrega d'emmagatzemar dins un ArrayList que es declara com atribut, objectes de tipus Soci. També es declara una variable de tipus Integer per poder establir un màxim de socis, en cas de que l'usuari vulgui. És per això que la classe presenta dos constructors. El constructor per defecte que ens indica que el nombre màxim de socis és 100 i un altre constructor que indica que el màxim de socis serà el nombre que ens indiqui el propi usuari.

La finalitat d'aquesta classe es poder gestionar tota la informació dels socis de manera eficient. Per fer-ho existeixen diferents mètodes que s'han declarat prèviament a la interfície (a la qual implementa LlistaSoci) InListSoci. Aquesta classe a més també implementa a la interfície Serializable per poder guardar objectes al fitxer.

Entre els mètodes implementats trobem mètodes com ara, getSize() que ens retorna la mida de l'ArrayList. El mètode addSoci(Soci soci) que afegeix els socis a l'ArrayList, en cas de que aquest no estigui ple ni el soci que es vol afegir ja hi sigui a la llista. Si qualsevol d'aquestes opcions succeeix es llença una excepció. En cas de voler treure un soci de la llista, trobem el mètode removeSoci(Soci soci) que crida a un mètode propi dels ArrayList que recorre tot l'ArrayList, en cas de ser-hi l'elimina i retorna un boolean true, i en cas de no ser-hi retorna un boolean false. D'aquesta manera aconseguim que si ens retorna un false poguem llançar una excepció conforme el soci que es vol eliminar no està a la llista. A més a més, també trobem els mètodes

verificarSocis() que comprova que tots els socis introduïts a l'ArrayList són correctes, i el mètode llistarSocis(String tipus) que retrona un String amb tota la informació dels socis que es troben a la llista en funció del tipus.

D'altra banda trobem mètodes que ens retornen dades dels socis en vers l'ArrayList. Com ara getAt(int position), ens retorna l'objecte soci que es troba a una posició determinada, o el mètode getSoci(String dni), que ens retorna l'objecet del soci en funció del dni.

I per últim trobem mètodes per gestionar l'ArrayList, com ara clear() que buida l'ArrayList, isFull() que comprova si l'ArrayList és ple, isEmpty() que fa la comprovació contraria cridant a un mètode propi dels ArrayList.

Asseguranca: La classe Asseguranca conté tota la informació de les assegurances que corresponen als diferents socis. Per això, es declaren dos atributs que defineixen un objecte de tipus Asseguranca que són:

```
private String _tipusAsseguranca;

private float _preuAsseguranca;
```

Aquesta classe implementa els getters i els setters de tots dos paramètres, els quals s'inicialitzen al constructor de la propia classe. A més, també conté un mètode toString() que retorna un String amb tota la información de l'objecte instanciat.

Com a remarcxa, destacarem el fet que es necessari que per poder guardar objectes al fitxer de tipus socis, que contenen objectes de tipus Asseguranca, la classe implementi la interficie Serializable.

Federacio: La classe Federacio és anàloga a la classe Asseguranca, donc té els mateixos mètodes però en compte de crear un objecte de tipus Asseguranca crea un mètode de tipus Federacio que depèn de dos nous atributs, similars als de Asseguranca:

```
private String _nomFederacio;

private float _preuFederacio;
```

La classe té el mateix funcionament, en essència, que la classe, ja explicada, Asseguranca.

ExcepcioClub: ExcepcioClub és una classe filla, de la classe pertanyent a Java, Exception. Aquesta només presenta un constructor que crida al super (constructor de la classe Exception).

La classe ExcepcioClub s'encarrega de gestionar els errors, gestionant els missatges d'errors que llencem en els diferents mètodes mitjançant els throws que després són capturats fent servir el try-catch en el bucle principal.

2.2 Explicar com has declarat l'atribut de la classe ClubUB i perquè.

Els atributs de la classe ClubUb són els següents:

```
private String nom;
private final int PREU_EXCURSIO = 20;
private final int PREU_QUOTA_MENSUAL = 25;
private final float DESCOMPTE_EXCURSIO_FEDERATS = 0.2f;
private final float DESCOMPTE_QUOTA_FEDERATS = 0.3f;
private LlistaSocis llista;
```

A continuació es dona una breu explicació del perquè de la seva existència.

```
private String nom;
```

És necessari un atribut de tipus String que indiqui el nom del club, ja que posteriorment en la classe VistaClubUB es cridara a un mètode getNom(), alhora d'imprimir el menú principal de l'aplicació. Per tal de poder implementar aquest getNom(), necessitem declarar una variable del mateix i inicialitzar-la el constructor, amés de fer el getter corresponent per poder ser cridat.

```
private final int PREU_EXCURSIO = 20;
```

Per tal de poder calcular el preu de les excursions realitzades pel soci, necessitem declarar un atribut final, ja que és constant, no varia, que ens indiqui quin és el preu d'una excursió. Atribut utilitzat al mètode:

- mostrarFacturaTotal(String dni, int excursions)

```
private final int PREU_QUOTA_MENSUAL = 25;
```

En aquí passa el mateix que amb PREU_EXCURSIO. Per poder calcular la factura dels socis necessitem d'aquesta variable final.

- mostrarFacturaTotal(String dni, int excursions)

```
private final float DESCOMPTE_EXCURSIO_FEDERATS = 0.2f;
```

Per tal de poder calcular correctament el preu de les excursions dels socis federats haurem de declarar una variable final que serà cridada des dels mètodes que requereixin d'aquesta. Com ara:

- crearSociFederat()

```
private final float DESCOMPTE_QUOTA_FEDERATS = 0.3f;
```

Amb la finalitat de calcular correctament el preu de la quota dels federats, n'és necessària la declaració d'aquest atribut. El mètode que depèn d'aquesta variable és:

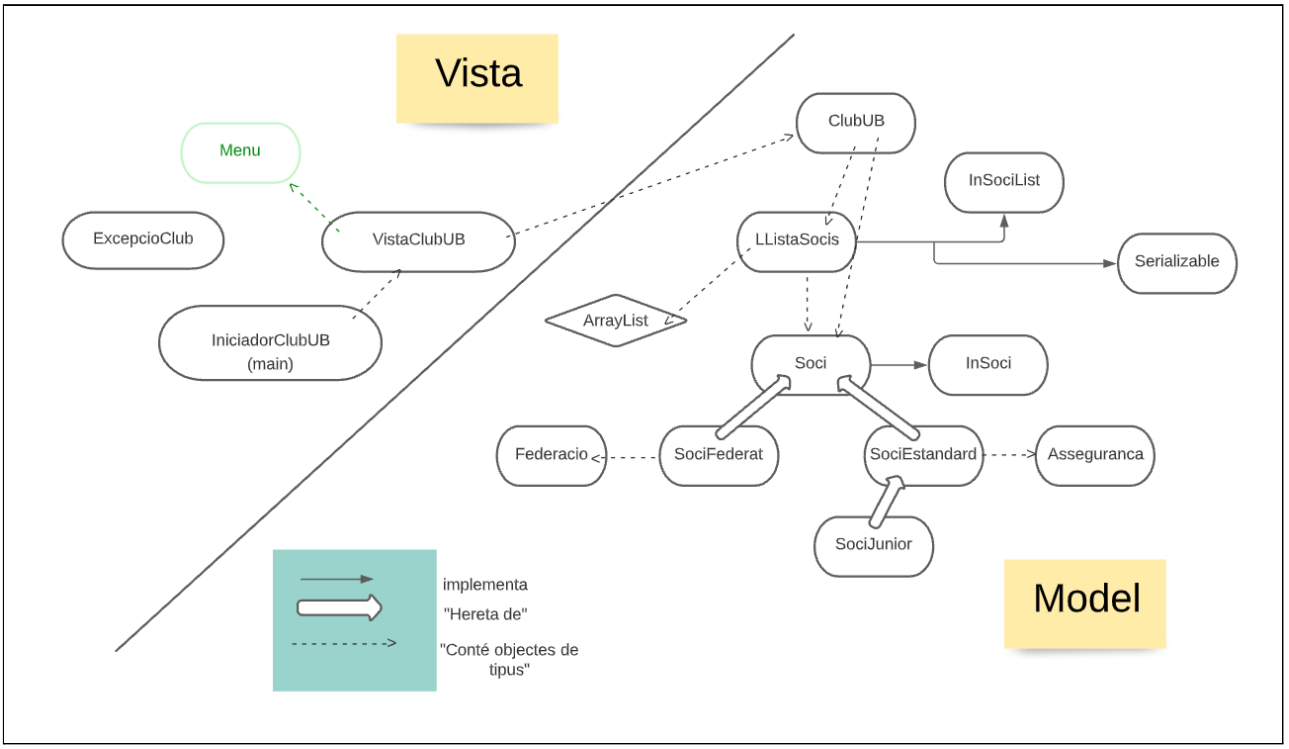
- crearSociFederat()

```
private LlistaSocis llista;
```

En el cas de llista és una variable de tipus LlistaSocis la qual conté un atribut ArrayList anomenada _socis que conté objectes de tipus Soci. Aquest atribut es necessari per tal d'implementar els mètodes següents:

- addSoci(Soci soci)
- llistarSocis(String tipus)
- removeSoci(int position)
- verificarSocis()
- mostrarFacturaTotal(String dni, int excursions)
- modificarNomSoci(String dni, String nom)
- modificarTipusAsseguranca(string dni, string assegurança, float preuAsseguranca)
- clear()

2.3 Dibuixar el diagrama de relacions entre les classes que heu utilitzat a la vostra pràctica. No cal incloure la llista d'atributs i mètodes.



2.4 Explicar quins són els atributs de la classe Soci i perquè.

Els atributs de soci són els presentats a continuació:

```
protected String nom;
protected String dni;
```

Els atributs d'una classe són les característiques individuals que determinen les qualitats d'un l'objecte del món real dins una classe. Tots els atributs que hem posat a la classe Soci són necessaris per la implementació dels seus mètodes. Sabent això, ara analitzarem el perquè de cadascun d'ells per separat i posteriorment, donarem l'explicació de perquè no calen més.

protected String nom;
L'atribut nom és de tipus String (cadena de caràcters), tots els socis han de tenir un nom que els caracteritza. Aquest atribut és protected degut a que sinó les diferents subclasses no podrien accedir-hi.

protected String dni;
En el cas de dni passa el mateix que amb nom. Per tal de crear un soci es necessita d'un DNI que diferenciï a uns socis dels altres. Aquest també és de tipus String, ja que és una combinació de números i una lletra. Amb una justificació anàloga a la de l'atribut nom, justifiquem perquè DNI ha de ser protected.

Per poder definir un objecte de tipus Soci cal que aquest quedi caracteritzat per diferents atributs. El fet que Soci sigui una classe abstracta i tingui, per tant, classes filles que creen objectes de tipus "soci" diferents, obliga a la classe mare a només contenir aquells atributs que caracteritzen a tots els socis. És a dir que són comuns a totes les subclasses. És per això que té sentit que només hagi de contenir aquests dos atributs encara que existeixen més elements necessaris per definir un Soci d'un tipus concret.

2.5 Per què creieu que la classe Soci ha de ser abstracta?

Una classe abstracta és aquella que no s'instancia, sinó que s'utilitza com a classe base per l'herència. Té sentit que aquesta classe hagi d'actuar com a classe base de les classes SociEstandard i SociFederat doncs totes dues contenen els mateixos mètodes però amb diferent implementació. És a dir segons en quina classe filla ens trobem, el comportament del mètode varia, llevat de tenir la mateixa signatura i el mateix tipus de retorn. D'aquesta manera fem més eficaç el nostre codi reutilitzant una gran part del codi, doncs SociEstandard i SociFederat, tenen molts punts en comú llevat d'unes poques diferències explicades a l'apartat 2.1.

2.6 Creieu que podríem haver treballat només amb dues classes Soci i SociFederat? Per què creus que és necessari crear les tres classes Soci, SociEstandard, SociFederat i SociJunior?

Des del nostre punt de vista, creiem que realitzar la pràctica amb només la classe Soci i Soci Federat hagués estat més complicat, ja que hauríem d'haver tractat amb uns constructors amb molts if-else if i tractant amb excepcions. Per exemple, hauríem d'haver inclòs una variable tipus que diferenciés els tres tipus de socis. En el cas de ser un Soci Federat, la variable tipus seria "Federat" i es faria una instància de Federació.
En el cas de ser un Soci Estàndard, la variable tipus seria "Estàndard" i es faria una instància de Asseguranca. En el cas de ser un Soci Junior la variable tipus seria "Junior" , es faria una instància de Asseguranca i a més a més es demanarà l'any de naixement. Això és el que passaria en el cas dels constructors. Per tant, en els mètodes de classe Soci hi hauria d'haver condicionales en el cas de que el soci fos o bé Estàndard o bé Junior, ja que

el Soci Federat, per ser classe filla de Soci, té implementats els mateixos mètodes de diferent forma. Cal recordar que en el cas de Soci Estàndard, les excursions costen diners, mentres que el Soci Junior no paga per les excursions.

Per tant, el fet de tenir la classe Soci com a classe mare i tenir dos filles com són la classe SociFederat i la classe SociEstandard i aquesta última tenir com a filla la classe SociJunior, ens ajuda molt a la neteja del codi i a que aquest codi sigui més ordenat, ja que ens estalviaríem una quantitat considerable de condicionals i excepcions i podríem implementar de millor forma i amb més facilitat i robustesa el codi dels mètodes corresponents a la classe Soci, els quals part d'ells són implementats a les classes filles per ser mètodes del tipus Abstract. També en el cas de SociEstandard i SociJunior ens estalviaríem diferenciar-los pel fet de que un demana l'any de naixement i l'altre no.

2.7 Indiqueu si hi ha i on es troben els exemples de mètodes polimòrfics al vostre codi.

En el nostre codi observem que tenim una classe mare que és la classe Soci. Aquesta classe és abstracta, ja que en aquesta classe en tenim diversos mètodes abstractes, que són toString(), getTipus(), calculaQuota (floatQuotaBase), calculaPreuExcursio (float preuExcursioBase) i comprova(), els quals són implementats en les diferents classes filles. D'aquesta en deriven dos tipus de socis, que són la classe SociEstandard i SociFederat. En particular, degut a que SociJunior és classe filla de SociEstandard, tal i com s'ha explicat a l'apartat 2.1, aquesta hereta els mètodes i atributs de la seva classe mare n'afegeix de nous. Per tant, com aquestes classes derivades tenen en comú que tenen implementats aquests mètodes abstractes de la classe Soci, amb la diferència de que com cadascuna de les classes té una particularitat, llavors tenim que els 4 mètodes anomenats prèviament són polimòrfics, ja que en cada classe prèviament anomenada s'implementen de forma diferent, però mantenen el mateix nom, signatura i retorn.

2.8 Analitzeu perquè l'objecte de tipus Asseguranca es crea fora del constructor de la classe SociEstandard. Es podria instanciar a dins? Com canviaria el sentit de la implementació?

Tècnicament, és molt diferent. El fet de crear l'objecte dins del constructor de SociEstandard provoca que cada vegada es creÏ una nova assegurança. per cada vegada que es cridi al constructor de SociEstandard. Si el posem fora, seguim creant l'assegurança, però amb la diferència de que en comptes de que en aquest cas establim el tipus i el preu, no creem des del constructor.

2.9 Expliqueu com heu utilitzat la classe ExcepcioClub al vostre codi.

Aquesta classe, tal i com hem dit a l'apartat 2.1, ens serveix per gestionar els errors que puguin haver-hi al programa i en cas de produir-se, llançar una excepció d'aquesta classe filla, que rep un missatge i crida al constructor de

la classe Exception, mitjançant super(String missatge). Per tant, el que hem fet ha estat personalitzar les possibles excepcions que puguin haver succeït, tal que aquestes tinguin un missatge que el client pugui entendre i no un propi de Java. Per tant, per tal de que es puguin gestionar aquests errors amb la nostra classe d'excepcions, el que hem fet és posar en la declaració dels mètodes on necessitem llançar l'excepció el següent: "throws ExcepcioClub". D'aquesta forma quan es llançava una excepció es cridava al constructor de EXcepcioClub i en aquest constructor es cridava al super, enviant el String missatge que s'havia rebut per paràmetre. Cal destacar que en la classe ClubUB, com tenia mètodes on es cridava a un mètode de LlistaSocis mitjançant una instància d'aquesta classe, llavors en els mètodes de ClubUB també s'havia de posar el throws ExcepcioClub, tot i que l'excepció no es cridés des d'aquell mètode, sinó des del mètode del tipus de Soci corresponent, ja que al cap i a la fi, LlistaSocis és una classe que a partir d'un ArrayList gestiona els diferents socis, i per fer determiinades accions, es fa un bucle per recórrer tota la llista, en el que per cada instància de Soci, es crida al mètode corresponent, on si hi ha un error, es llença una excepció. Per tant, tant en els mètodes de ClubUB com en els de LlistaSocis, s'ha de posar el throws ExcepcioClub, perquè tots aquests depenen de les diferents classes de socis.

2.10 Si teniu un soci federat que fa 5 excursions en un mes quina serà la quantitat de la seva factura aquell mes?

El preu de la factura mensual es calcula de la següent manera:

$$factura = Quota_{mensual} \cdot \left(1 - Descompte_{mensual}\right) + Num_{excursions} \cdot Preu_{excursions} \cdot \left(1 - Descompte_{excursions}\right)$$

Si substituïm obtenim:

$$factura = 25 \cdot (1 - 0,3) + 5 \cdot 20 \cdot (1 - 0,2) = 97,5€$$

2.11 Si teniu un soci estàndard amb una assegurança de tipus Bàsica d'un preu de 5€ i que fa 5 excursions en un mes quina serà la quantitat de la seva factura aquell mes?

El preu de la factura mensual es calcula de la següent manera:

$$factura = Quota_{mensual} + Num_{excursions} \cdot Preu_{excursions}$$

$$factura = Quota_{mensual} + Num_{excursions} \cdot (Preu_{excursionsBase} + Preu_{assegurança})$$

Si substituïm obtenim:

$$factura = 25 + 5 \cdot (20 + 5) = 150€$$

2.12 Detallar les proves realitzades per comprovar el correcte funcionament de la pràctica, resultats obtinguts i accions derivades.

Per poder comprovar el correcte funcionament del programa vam fer un seguit de proves que ens va permetre corretgir els errors presents en aquest. En el procés de comprovació vam decidir seguir un esquema amb la fi de poder comprovar totes i cadascuna de les opcions en els seus casos limit.

Primer, varem comprobar la funcionalitat de la primera opció. En aquí vam observar un problema, el submenú s'imprimia bé, però en afegir-hi socis amb el mateix DNI no es llençava cap execepció. Ens vam adonar que era degut a la dolenta implementació del mètode *addSoci()* en la crida al mètode *contains*. Aquest mètode necessita d'un mètode *equals* (es troba a la classe Soci), al qual s'ha li ha de passar un *Object* per paràmetre. El mètode *equals* s'encarrega de comprovar si dos socis diferents tenen el mateix DNI, en fer-ho, l'*Object* que havíem passat per parametre passa a ser un objecte de tipus *Soci*. El nostre error era que passavem un objecte de tipus *Soci* directament i, per tant això no permetia el correcte funcionament del mètode *contains*. En canviar-ho el mètode contains ja funcionava perfectament i llença l'execpció corresponent en cas de trobar un altre soci amb el mateix DNI dins la llista de *clubUB*.

Posteriorment, i un cop l'opció 1 funcionava correctament, vam afegir 2 socis de cada tipus per poder comprovar la resta d'opcions del menú principal. En aquest moment varem seleccionar les opcions 2, 3, 4 i 5 les quals funcionaven perfectament mostrant les llistes de socis demanades. Per fer la comprovació de l'opció 6, "Eliminar soci", varem introduir una posició que es trobava dins la llista i per verificar que s'havia eliminat correctament, vam fer servir l'opció dos, mostrant tota la llista de Socis inscrits. En aquesta opció, a més a més vam veure que l'excepcion també es llençava correctament en cas de voler esborra una posició que no fos a la llista.

En el cas de l'excepció 7, la comprovació va consistir a inscriure socis amb dades errònies com és el cas del preu de la federació o el tipus d'assegurança. En tots dos casos es llença correctament les respectives exepcions.

En el cas de les opcions 8, 9 i 10 vam seguir el mateix procediment. El seu funcionament era correcte. En el cas de les opcions 11 i 12, vam haver d'inscriure més socis a la llista. Després vam cridar a l'opció 11 i vam tancar sessió, cridant a l'opció 13 la quan funcionava a l aperfecció. Llavors, vam fer debugging un altre cop i vam tornar a imprimir la llista de socis per veure que no hi hagués cap inscrit. Un cop feta aquesta comprovació vam cridar a l'opció 12 i després a l'opció 2 on van aparèixer els socis que haviem inscrit en la sessió anterior.

Amb totes aquestes proves, vam veure que el nostre programa funcionava perfectament llençant totes les excepcions demanades a l'enunciat.

2.13 Observacions generals

Veiem que el programa es composa d'una gran quantitat de classes que fan que el sistema pugui funcionar de la forma desitjada. Hem vist anteriorment com estan fetes les classes i el funcionament de cadascuna.

Per una part, trobem a la Vista tres classes essencials:

- IniciadorClubUB: És la classe que conté el main del programa i que s'encarrega de crear objectes de tipus VistaClubUB, que fa que aquest cridi al mètode gestióClub(), el qual dins d'aquest té el mètode mostrarMenu() de la classe Menu.
- Menu: s'encarrega de que el menú on es mostren totes les funcionalitats del programa funcioni correctament.
- VistaClubUB: Aquesta classe té la responsabilitat de comunicar el programa i a l'usuari. Aquest conté un atribut privat del tipus ClubUB que conté tota la informació sobre el club UB. Trobem que aquest mètode, alhora depèn de la classe Menu per poder inicialitzar el menú correctament.

Per altra banda, tenim la part de Model les següents classes:

- ClubUB: Aquesta classe és essencial pel programa, ja que conté tota la informació del club i té la instància de la classe LlistaSocis que permet accedir a la informació dels socis del club.
- Soci: És la classe mare de SociFederat i SociEatandard i conté els atributs i mètodes més generals dels diferents tipus de soci, és a dir, els atributs i mètodes implementats que tenen en comú totes les classes filles i declarats els mètodes abstractes per la seva implementació (aplicant polimorfisme) a les diferents classes.
- SociFederat: És una classe filla de Soci i com a tal hereta atributs i mètodes, a part d'implementar els mètodes abstractes de Soci. Com a particularitat, té un atribut de la classe Federació, exclusiu d'aquesta classe SociFederat.
- SociEstandard: És una classe filla de Soci i com a tal hereta atributs i mètodes, a part d'implementar els mètodes abstractes de Soci. Com a particularitat, té un atribut de la classe Asseguranca, exclusiu d'aquesta classe SociEstandard i de SociJunior, per ser aquesta filla de SociEstandard.
- SociJunior: És la classe néta de Soci i per tant la classe filla de SociEstandard. Com a tal, hereta tots els atributs heretats de classe SociEstandard i els propis de SociEstandard, a més a més de l'atribut de l'any de naixement, propi de la seva classe.
- LlistaSocis: És la classe que conté, a partir d'un ArrayList, tots els socis que són del club. Aquesta classe és essencial per tal d'accedir a tots els socis o algun soci en concret. Per això, aquesta classe conté mètodes per realitzar comprovacions o trobar un soci en concret, ja que es realitza un bucle per anar cridant a la funció corresponent de l'element de la llista, en aquest cas d'un soci.

- Asseguranca: Aquesta classe serveix per gestionar l'atribut propi de SociEstandard i SociJunior, que és l'assegurança.
- Federació: Aquesta classe serveix per gestionar l'atribut propi de SociFederat, que és la federació.
- ExcepcioClub: Classe filla de Exception (pròpia de Java) que serveix per gestionar les excepcions del programa.

Observem que aquestes classes tenen una relació molt estreta, ja que cadascuna té instàncies de les altres classes, i depenen d'aquestes pel seu funcionament correcte.

3. Resultats.

El resultat d'aquesta tasca ha estat un programa que un cop executat es mostra per pantalla un menú que imprimeix les opcions que es mostren a continuació:

```
-----------------------------------------------
CLUB UB
-----------------------------------------------
        1.- Donar d'alta un soci
        2.- Mostrar llista de socis
        3.- Mostrar llista de socis federats
        4.- Mostrar llista de socis estàndards
        5.- Mostrar llista de socis junior
        6.- Eliminar soci
        7.- Verificar socis
        8.- Mostrar total factura
        9.- Modificar nom soci
        10.- Modificar tipus assegurança soci
        11.- Guardar dades
        12.- Recuperar dades
        13.- Sortir
-----------------------------------------------
Entra una opcio >> 
```

Si entrem una opció vàlida, podem observar que el programa realitza les tasques corresponents correctament. Per verificar això, vam fer un conjunt de proves que s'han detallat en l'apartat 2.12 en cadascuna de les opcions del menú. Per això, una vegada s'iniciava el programa, vam escollir una de les opcions [1, 13] i fer les proves corresponents. En particular, tenim que a l'opció 1 del menú, corresponent a donar d'alta un soci, se'ns obre un submenú, ja que el soci que podem afegir pot ser de diferents tipus. El submenú en qüestió és el següent:

```
-------------------------------------
Entra una opcio >> 1
-----------------------------------
INDICA UNA DE LES SEGÜENTS OPCIONS
-----------------------------------
        1.- Afegir Soci Federat
        2.- Afegir Soci Estàndard
        3.- Afegir Soci Junior
        4.- Menú anterior
-----------------------------------
Entra una opcio >> 
```

Llavors, per cada opció d'aquest submenú, es va comprovar si funcionava cada tipus de soci i si l'opció de retorn també.

Per tant, un cop vam realitzar tot això, vam poder comprovar que el programa funcionava en la seva totalitat, ja que feia totes les accions demanades, determinant així una possible implementació d'un menú d'un club i la gestió dels seus socis.